



Common Patterns

Graph Search & Pathfinding

BFS, DFS, Dijkstra's, and A* — why graph traversal shows up constantly in AI coding problems and how to recognize when you need it.

Graph problems are everywhere in AI coding interviews: navigation between waypoints, network packet routing, dependency resolution, social network analysis, and game state exploration. If you zoom out, a surprising number of problems reduce to "find a path through a graph," and from there the solution follows directly.

Your trusty AI can write any of these algorithms in seconds. What interviewers care about is whether you can spot that a problem is a graph problem in the first place, pick a sensible traversal, explain to them what the algorithm is doing, and steer the AI through the back-and-forth without getting lost.

Recognizing the pattern

Many graph problems hide their structure inside a described process: finding the shortest sequence of word transformations, navigating a maze, propagating a state through a network. The interviewer is watching whether you can see the graph underneath.

Tells that you're looking at a graph problem:

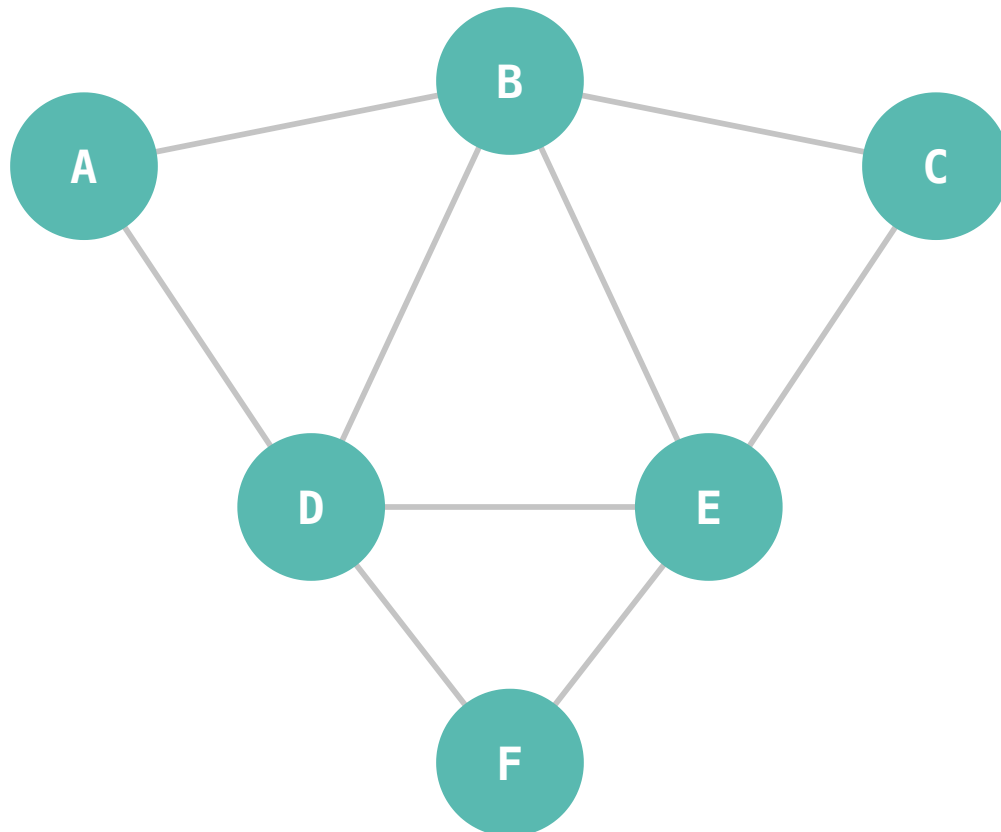
- **"Things" and "connections between things."** Any time the input has entities and a relationship between them (people and friendships, cities and roads, courses and prerequisites, files and imports), you can model it as a graph. The question is whether you should.
- **Shortest, fewest, minimum, or "any path between."** These are pathfinding questions. Add weights to the edges and you're in Dijkstra's territory; without weights, BFS.
- **Reachability or connectivity.** "Can X reach Y?" "How many components are there?" "Is this network fully connected?" These are traversal questions, so BFS or DFS.
- **2D grids with movement rules.** A grid is a graph in disguise where each cell connects to its 4 or 8 neighbors. Almost every "rooms," "islands," "maze," or "minimum steps" grid problem reduces to BFS or DFS on this implicit graph.

- **A sequence of transformations.** Word ladders, state-space search, game playing. Each state is a node; each legal transformation is an edge. Once you frame it that way, the algorithm choice follows.

When you spot one, say it out loud before you start coding: "This is a shortest-path problem on an unweighted graph, so I'm thinking BFS." That's the signal interviewers are reading for.

What is a graph?

A graph is a collection of nodes (sometimes called vertices) connected by edges. That's it. If you can model something as "things" and "connections between things," you've got a graph.



The fact that *so many problems* can be modelled as a graph is a feature, not a bug. There are a lot of problems that are not obviously graphs that can be modeled as graphs, so you'll get a lot of mileage out of this pattern.

There are two standard ways to represent a graph in code, and you should know both because the AI will ask (or assume) one or the other.

Adjacency list. Each node stores a list of its neighbors. This is the default for most interview problems because it's memory-efficient for sparse graphs, which covers most real-world graphs and keeps test cases with large graphs from requiring huge binaries.

Adjacency List

Python 

```
graph = {  
    "A": ["B", "D"],  
    "B": ["A", "C", "D", "E"],  
    "C": ["B", "E"],  
    "D": ["A", "B", "E", "F"],  
    "E": ["B", "C", "D", "F"],  
    "F": ["D", "E"],  
}
```

Adjacency matrix. A 2D grid where `matrix[i][j] = 1` means there's an edge from node `i` to node `j`. This is better for dense graphs or when you need $O(1)$ edge lookups, but uses memory.

Adjacency Matrix

Python 

```
#      A  B  C  D  E  F  
matrix = [  
    [0, 1, 0, 1, 0, 0], # A  
    [1, 0, 1, 1, 1, 0], # B  
    [0, 1, 0, 0, 1, 0], # C  
    [1, 1, 0, 0, 1, 1], # D  
    [0, 1, 1, 1, 0, 1], # E  
    [0, 0, 0, 1, 1, 0], # F  
]
```

In practice, you'll almost always use adjacency lists. If the AI generates a matrix representation and the graph is sparse, that's a red flag worth catching.



When you encounter a new problem in the interview, don't immediately think about algorithms. First ask: "Can I model this as a graph?" If yes, figure out what the nodes are, what the edges are, and whether edges have weights. The algorithm choice follows from that.

Junior candidates often only spot graphs when they're literally in the prompt: "social network", or "dependency graph". But a lot of the power of a graph representation is that they apply to things more abstract things like game-playing where the edges might represent possible moves. You'll need to be creative in spotting them!

Graph traversal algorithms

Four algorithms cover almost everything you'll see: BFS and DFS for unweighted graphs, Dijkstra's for weighted graphs, and A* when you have a way to estimate distance to the goal. The right pick falls out of the problem shape, so it's worth knowing what each one buys you.

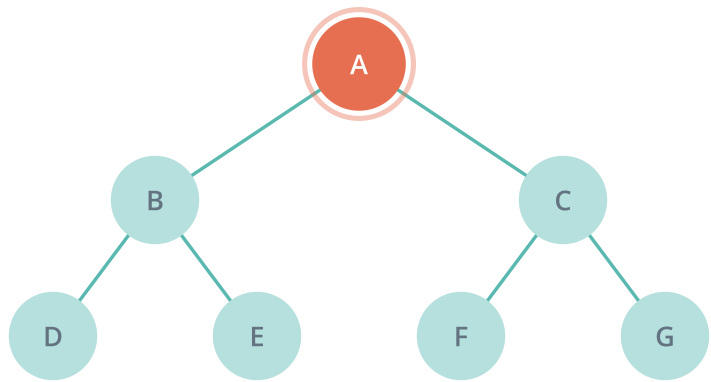
BFS: Breadth-First Search

BFS explores a graph level by level. It visits all neighbors of the starting node first, then all neighbors of those neighbors, and so on. Think of it like dropping a stone in a pond: the ripple expands outward uniformly.

This property makes BFS the go-to algorithm for finding the shortest path in an unweighted graph. Because it explores nodes in order of their distance from the start, the first time it reaches any node is guaranteed to be via the shortest path.

```
bfs(root)
queue = [root]

while queue not empty:
    node = queue.shift()
    visit(node)
    queue.add(node.left)
    queue.add(node.right)
```



Starting from the root, BFS pulls one node off the queue, marks it visited, and pushes its unvisited neighbors to the back. Nodes finish in level order: depth-1 before depth-2, depth-2 before depth-3.

Breadth-First Search

Python  

```
from collections import deque

def bfs(graph, start, target):
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        node, path = queue.popleft()
        if node == target:
            return path

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None
```

The key data structure is the **queue** (FIFO). Nodes go in at the back and come out at the front, which enforces the level-by-level ordering. The **visited** set prevents cycles from causing infinite loops.

When to use BFS:

- Shortest path in an unweighted graph
- Level-order traversal (e.g., "find all nodes within K hops")
- Any problem that asks for the *minimum* number of steps

DFS: Depth-First Search

DFS takes the opposite approach. Instead of exploring broadly, it goes as deep as possible down one path before backtracking. It's like exploring a maze by always taking the first turn you see and only backing up when you hit a dead end.

DFS (Recursive)

Python  

```
def dfs(graph, start, target, visited=None):
    if visited is None:
        visited = set()

    visited.add(start)
    if start == target:
        return [start]

    for neighbor in graph[start]:
        if neighbor not in visited:
            result = dfs(graph, neighbor, target, visited)
            if result:
                return [start] + result

    return None
```

You can also write DFS iteratively with an explicit stack, which avoids recursion depth limits for large graphs:

DFS (Iterative)

Python  

```
def dfs_iterative(graph, start, target):
    stack = [(start, [start])]
    visited = set()
```

```
while stack:
    node, path = stack.pop()
    if node == target:
        return path

    if node in visited:
        continue
    visited.add(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            stack.append((neighbor, path + [neighbor]))
```

DFS doesn't guarantee shortest paths. The path it finds depends entirely on the order it explores neighbors. So why use it?

When to use DFS:

- **Cycle detection.** As DFS walks down a path, it can mark nodes as "currently on the stack." Bumping into one of those again means you've looped back, which is a cycle.
- **Topological sorting** (see the [topological sort](#) article). Finishing order from DFS naturally produces a valid ordering of nodes that respects dependencies.
- **Finding *any* path** (not necessarily shortest). When you just need *a* solution, going deep is often faster than expanding the whole frontier level by level.
- **Exploring all possible paths** (e.g., maze solving, backtracking problems). DFS pairs naturally with the choose/explore/unchoose rhythm because the recursion stack already tracks where to back up to.
- **Connected component detection.** Start DFS from each unvisited node; everything you touch is one component. Repeat until every node has been seen.
- **Problems with deep recursive structure.** If the problem itself is recursive (trees, nested expressions, fractal-like state), DFS's recursion mirrors the structure for free.

DFS uses less memory than BFS in practice. BFS has to store the entire frontier (which can be huge for wide graphs), while DFS only stores the current path. For very large graphs, this matters.



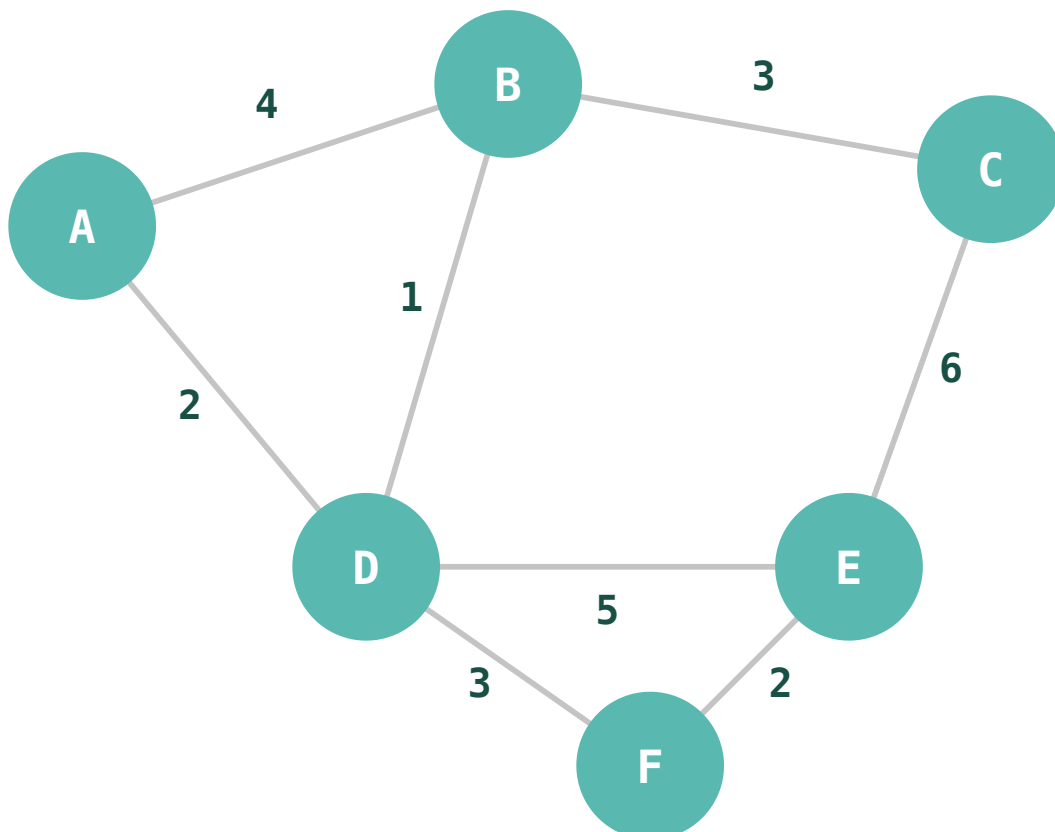
A quick heuristic: if the problem asks for "shortest" or "minimum," reach for BFS. If it asks for "all paths," "any path," or involves backtracking, reach for DFS.

The reason BFS gives you shortest paths for free is that it processes nodes in order of distance from the start. The first time it reaches any node, that has to be by the fewest edges. DFS makes no such promise; it commits to a direction until it bottoms out, so the path it finds first is usually a long detour.

When in doubt, BFS is the safer default.

Weighted graphs and Dijkstra's algorithm

So far we've assumed every edge has the same weight. Real problems often don't work that way. In a road network, different roads have different distances. In a network, different links have different latencies. When edges have weights, BFS no longer gives you the shortest path; it gives you the path with the fewest *edges*, not the lowest *total weight*.



Enter Dijkstra's algorithm. It's essentially BFS, but instead of a regular queue, it uses a priority queue (min-heap) that always processes the node with the smallest known distance first. This guarantees that when you reach a node, you've found the cheapest path to it.

Dijkstra's Algorithm

Python  

```
import heapq

def dijkstra(graph, start, target):
    heap = [(0, start, [start])]
    visited = set()

    while heap:
        cost, node, path = heapq.heappop(heap)

        if node == target:
            return cost, path

        if node in visited:
            continue
        visited.add(node)

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                heapq.heappush(heap, (cost + weight, neighbor, path + [neighbor]))
```

The graph representation changes slightly for weighted graphs. Instead of a list of neighbors, each entry is a tuple of `(neighbor, weight)`:

Weighted Graph

Python  

```
graph = {
    "A": [("B", 4), ("D", 2)],
    "B": [("A", 4), ("C", 3), ("D", 1)],
    "C": [("B", 3), ("E", 6)],
    "D": [("A", 2), ("B", 1), ("E", 5), ("F", 3)],
    "E": [("C", 6), ("D", 5), ("F", 2)],
    "F": [("D", 3), ("E", 2)],
}
```

In this graph, the shortest path from A to C isn't the direct A-B-C route (cost 7). It's A-D-B-C (cost 6), because the detour through D is cheaper. Dijkstra's finds this automatically.



The key skill is recognizing when a problem is a weighted shortest-path problem and telling the AI to use Dijkstra's. You'll be looking for the priority queue or heap. Then verify the output has the right structure: a heap, a visited set, and distance tracking.

Your interviewer wants to see some signals that you know what you're looking at, not accepting any implementation as passing.

A*: Dijkstra's with a heuristic

A* is Dijkstra's smarter cousin. The insight for it is pretty simple: if you have some idea of which direction the target is in, you can prioritize nodes that seem closer to the goal. Instead of sorting the heap by `cost_so_far`, A* sorts by `cost_so_far + estimated_cost_to_goal`.

That estimated cost is the heuristic function, often called $h(n)$. For grid-based pathfinding, Manhattan distance or Euclidean distance are common choices. As a concrete example: if you're at grid cell $(2, 3)$ and the goal is $(5, 7)$, the Manhattan distance is $|5-2| + |7-3| = 7$. You don't actually know if there's a clear path that short (there could be walls), but you know any real path has to be *at least* that long. That's the property A* uses to decide which frontier node to expand next.

The heuristic has to be *admissible*: it can never overestimate the actual cost. If it does, A* might rule out the optimal path before ever exploring it. Manhattan distance is admissible on a 4-directional grid because the real path can never be shorter than walking straight to the goal.



Problems with heuristics show up a lot in AI coding interviews in part because there's often no "book" answer. A* is a great example of a way to use these heuristics, and they'll show up in other approaches like branch and bound which we'll cover in the [backtracking](#) pattern.

This basically means A* is most useful in scenarios where there is some *structure* to the problem that can be used to estimate the cost to the goal like if the problem involves spatial pathfinding (grid navigation, map routing, robot movement). In these cases, A* will be significantly faster than plain Dijkstra's because it avoids exploring nodes in the wrong direction. For abstract graphs without spatial structure, A* degrades to Dijkstra's because there's no useful heuristic.

A* Search

Python  

```
import heapq

def astar(graph, start, target, heuristic):
    heap = [(heuristic(start, target), 0, start, [start])]
    visited = set()

    while heap:
        _, cost, node, path = heapq.heappop(heap)

        if node == target:
            return cost, path

        if node in visited:
            continue
        visited.add(node)

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                new_cost = cost + weight
                priority = new_cost + heuristic(neighbor, target)
```

For a 2D grid where nodes have coordinates:

Heuristic Functions

Python  

```
def manhattan_distance(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def euclidean_distance(a, b):
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5
```



If you have a heuristic or an estimate for the cost to the goal, you should probably be thinking about using A*.

Prompting the AI

The move on graph problems is to tell the interviewer the algorithm and tell the AI the goal. "This is an unweighted shortest path problem, so I'm thinking BFS" is what the interviewer needs to hear. Five seconds, and it demonstrates real understanding. When you prompt the AI, leave a little room: "find the shortest path between these nodes" is better than locking it to BFS, because the AI might suggest something better suited to the specifics, and you want to keep that door open.

A few pattern-specific moves:

- **For grid problems, name the reframe.** Tell the AI "treat the grid as a graph where each cell connects to its 4 neighbors" rather than asking for a specialized grid traversal. You get a standard BFS/DFS that's easier to read and verify.
- **For weighted graphs, confirm the edge weights.** "Weights are non-negative, so Dijkstra's works" is a useful sentence to say, both to the interviewer and to the AI. If you skip this and there's a negative edge, Dijkstra's will silently produce a wrong answer.
- *For heuristic-based search (A), ask the AI to generate a harness to validate the heuristic.** The AI is much better at writing heuristics than verifying them. A small adversarial test set is a high-leverage prompt.



If you're uncertain which algorithm fits, say so. "I think this is a shortest-path problem with weighted edges, so Dijkstra's should work. Let me verify that the weights are all non-negative." This kind of transparent reasoning scores better than silently picking an algorithm and hoping it's right.

Verifying the AI's code

Graph algorithms are short and well-known, so the AI usually gets the structure right. The bugs live in a few specific places:

1. **BFS that doesn't mark visited on enqueue.** Marking visited on dequeue is a classic mistake; you'll add the same node to the queue multiple times and either get an infinite loop or a wrong distance. Check the line where `visited.add(node)` lives. It should be right after appending to the queue, not after popping from it.
2. **Dijkstra's without the "already-visited" skip.** A correct implementation pops a node from the heap and immediately checks whether it's been processed; if so, skip. Without this, you re-process nodes and get the wrong distance on graphs with multiple paths to the same node.
3. **DFS used for shortest path.** If the AI generated DFS for a "minimum steps" problem, that's a structural bug. DFS will time out on anything non-trivial and may not find the shortest path at all.
4. **Edge cases the test set didn't surface.** Disconnected graphs (target might be unreachable, so does the code return a sentinel or crash?), self-loops, duplicate edges, the graph being a tree (no cycles to handle). The provided test data may not exercise these, but mentioning them out loud tells the interviewer you're thinking past the happy path.

The fastest sanity check: name the data structure. BFS uses a queue, DFS uses a stack (or recursion), Dijkstra's uses a min-heap, A* uses a min-heap sorted by $f = g + h$. If the AI's code disagrees with what the algorithm name implies, something is wrong before you even read the loop.

When to use vs. alternatives

Algorithm	Time	Space	Use when
BFS	$O(V + E)$	$O(V)$	Unweighted shortest path, level-order traversal, minimum steps
DFS	$O(V + E)$	$O(V)$	Cycle detection, topological sort, exhaustive enumeration, connected components

Algorithm	Time	Space	Use when
Dijkstra's	$O((V + E) \log V)$	$O(V)$	Weighted shortest path with non-negative weights
A*	$O((V + E) \log V)$	$O(V)$	Spatial pathfinding with an admissible heuristic

Graph time complexities are hairy and "bookish"; interviewers don't usually drill on them, but knowing the rough shape helps you flag when a solution will time out. BFS and DFS have the same theoretical complexity, but their memory profiles differ in practice (BFS stores the full frontier; DFS only stores the current path). Dijkstra's and A* have the same worst case, but A* is faster in practice when the heuristic is good.

Neighbor patterns that get confused with graph search:

- **Topological sort** is graph search restricted to DAGs where the *order* of traversal is the answer. If the problem is "in what order can these tasks run?", reach for topo sort, not generic graph search.
- **Negative-weight shortest paths** break Dijkstra's. Use Bellman-Ford instead. If the problem explicitly mentions negative weights (cashback, refunds, currency exchange), don't reach for Dijkstra's.
- **Backtracking** comes up when the problem is "find *all* paths" or "enumerate all valid configurations" rather than "find *a* path." DFS is the engine underneath, but the framing is different.

What interviewers expect

Beyond the recognition step (which is the biggest single signal), interviewers are watching for:

- **You can explain the algorithm the AI produced.** "It's BFS, but with a tuple of (node, distance) instead of a separate distance map" is the kind of narration that tells the interviewer you understood the choice. If you can't competently explain it, they can't tell whether you got it right or got lucky.
- **You proactively name the edge cases.** Disconnected graphs, self-loops, duplicate edges, the graph being a tree (no cycles). These may not even appear in the provided test data,

but flagging them shows completeness, and they're easier to spot once you've recognized the graph in the first place.

- **You frame the algorithm choice as a decision, not a default.** "I'm using Dijkstra's because the edges are weighted and non-negative" beats "I'm using Dijkstra's because shortest path." Stating the precondition out loud is the signal.
- **You catch the high-leverage bugs.** Mark-on-enqueue for BFS, the already-visited skip for Dijkstra's, DFS-for-shortest-path. Naming one of these out loud while reading the AI's code is the kind of thing strong candidates do.

Putting it together

Graph search is one of those patterns where recognition matters far more than implementation. In an AI coding interview, the AI can generate BFS, DFS, or Dijkstra's in seconds. What it can't do is answer the question from the interviewer before hands-on-keyboard to realize that "minimum number of bus transfers to reach a destination" is a BFS problem on a graph where bus routes are nodes.

That insight is yours to bring. Practice spotting graph structure in problems that don't obviously look like graph problems. When you see one, name the algorithm, explain your reasoning to the interviewer, and let the AI handle the boilerplate. Then verify the output against the patterns you know: queues for BFS, stacks for DFS, heaps for Dijkstra's and A*. That's the workflow interviewers want to see.

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned and identify gaps.

 Start Quiz